



南方科技大学

LAB 5: Attention Mechanism

COE 201 - AI Large Models, Spring 2026

2026-05-17

College of Engineering, SUSTech

Office Hour: Thursday, 14:30-15:30 @ 3rd Research Building, Room 610

Outline

The Problem with RNNs	1
Understanding Attention	3
Multi-Head Attention	12
Lab Tasks	19

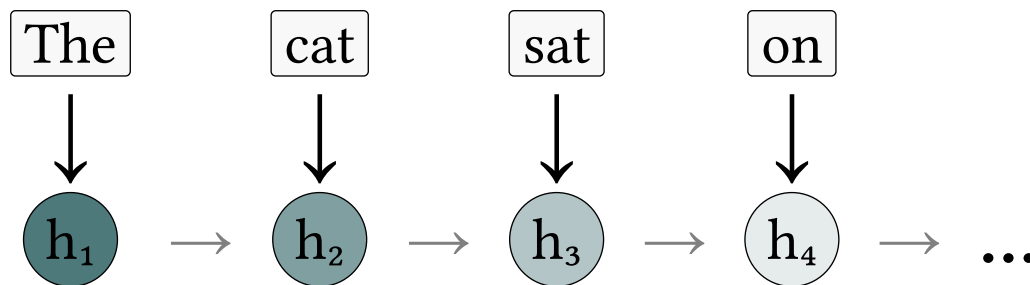
The Problem with RNNs

Limitations of Sequential Processing



南方科技大学

RNNs process sequences **one step at a time**:



Color fade = Information from “cat” gets diluted over time

Problem: Information from early steps gets **diluted** as it passes through hidden states.

Example: “The cat, which was hungry and had not eaten for hours, sat on the mat”

- By the time we reach “sat”, does the model still remember “cat”?

Solution → **Attention Mechanism**

Understanding Attention

The Search Engine Metaphor



Attention works like a **search engine**:

Concept	Analogy	In Attention
Query	What you're looking for	Current word asking for context
Key	Index for lookup	All words' "identifiers"
Value	Actual content	All words' actual meanings

Intuition: For each word (Query), we search through all words (Keys) to find relevant ones, then retrieve their content (Values).

Sentence: “The **cat** sat on the mat”

When the word “**sat**” is the Query:

Query

“**sat**” → “*Who did the action?*”

Keys

“The” “**cat**” “**sat**” “on” “the” “**mat**” → *Each word’s id*

Values

Meaning vectors for each word

Result: “sat” attends most to “**cat**” (subject) and “**mat**” (object) — the Keys for “cat” and “mat” had the highest similarity to the Query for “sat”.

Linear Projection to Q, K, V



We apply learned **Linear Projections** (weight matrices) to the input X :

$$\begin{array}{lcl} \begin{array}{c} X \\ \text{(batch, seq, d_model)} \end{array} & \times & \begin{array}{c} W^Q \\ \text{(d_model, d_k)} \end{array} = \begin{array}{c} Q \\ \text{(batch, seq, d_k)} \end{array} \\ \begin{array}{c} X \\ \text{(batch, seq, d_model)} \end{array} & \times & \begin{array}{c} W^K \\ \text{(d_model, d_k)} \end{array} = \begin{array}{c} K \\ \text{(batch, seq, d_k)} \end{array} \\ \begin{array}{c} X \\ \text{(batch, seq, d_model)} \end{array} & \times & \begin{array}{c} W^V \\ \text{(d_model, d_v)} \end{array} = \begin{array}{c} V \\ \text{(batch, seq, d_v)} \end{array} \end{array}$$

- X : Input sequence embeddings, shape: (batch, seq, d_model)
- W^Q, W^K : Learned weights, shape: (d_model, d_k); W^V shape: (d_model, d_v)

Linear Projection to Q, K, V



- Q, K : Projected tensors, shape: (batch, seq, d_k); V shape: (batch, seq, d_v)
 - Q, K **must** be in same shape with a hidden size d_k (to perform dot products)
 - V are not required to be in same hidden shape d_k , but normally: $d_k = d_v$

The core attention formula:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

1. **Compute relevance:** $QK^T \rightarrow$ “How relevant is each key to my query?”
 2. **Normalize:** Softmax $\rightarrow$ “Convert to probabilities”
 3. **Weighted sum:** Multiply by $V \rightarrow$ “Aggregate relevant values”
- d_k : Used for **matching** (similarity) between Query and Key.
 - d_v : Used for the actual **content** (Value), normally $d_v = d_k$, so we only use d_k for represent shape.

Why Scale by $\sqrt{d_k}$?



Problem: As dimension d_k increases, dot products grow larger:

- For $d_k = 64$: dot products are small $\rightarrow$ softmax gradients are stable
- For $d_k = 512$: dot products become huge $\rightarrow$ softmax saturates

Softmax Saturation:

1	Input: [1, 2, 3, 4]	$\rightarrow$ softmax: [0.03, 0.09, 0.24, 0.64]	✓ Gradients flow
2	Input: [10, 20, 30, 40]	$\rightarrow$ softmax: [0.00, 0.00, 0.00, 1.00]	✗ Gradients vanish!

Scaling by $\sqrt{d_k}$ keeps dot products in a reasonable range.

Attention Matrix Visualization



Each row shows what a word “attends to” (pays attention to).

	The	cat	sat	on	the	mat
The	0.3	0.2	0.2	0.1	0.1	0.1
cat	0.1	0.4	0.2	0.1	0.1	0.1
sat	0.1	0.3	0.3	0.1	0.1	0.1
on	0.1	0.1	0.2	0.4	0.1	0.1
the	0.1	0.1	0.1	0.1	0.5	0.1
mat	0.1	0.1	0.1	0.1	0.1	0.5

Key property: Every word can directly attend to **any** other word — no sequential bottleneck!

Causal (Masked) Attention



Problem: When generating text, we can't look at future tokens!

Solution: Apply a **causal mask** (lower-triangular matrix):

	The	cat	sat	on	the	mat
The	1	0.0	0.0	0.0	0.0	0.0
cat	0.4	0.6	0.0	0.0	0.0	0.0
sat	0.3	0.3	0.4	0.0	0.0	0.0
on	0.2	0.2	0.3	0.3	0.0	0.0
the	0.1	0.1	0.2	0.2	0.4	0.0
mat	0.1	0.1	0.1	0.1	0.2	0.5

Grey area = Future tokens (masked out during training/inference)

Implementation: Set upper triangle to $-\infty$ before softmax (they become 0).

Multi-Head Attention

Why Multiple Heads?



Problem: A single attention head can only capture **one type** of relationship.

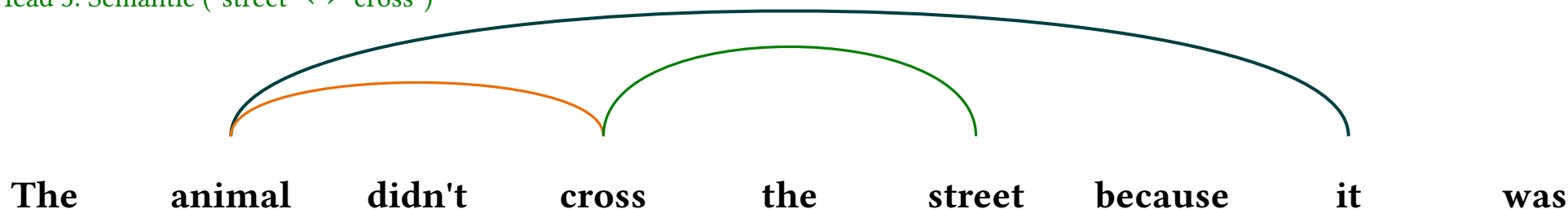
Example: “The animal didn’t cross the street because it was too tired”

Different heads can learn different relationships:

Head 1: Pronoun (“it” → “animal”)

Head 2: Syntax (“animal” → “cross”)

Head 3: Semantic (“street” ↔ “cross”)



Multi-Head = Multiple “perspectives” looking at the same sequence simultaneously.

How do we actually implement these “multiple perspectives”?

→ use multiple sets of W^Q, W^K, W^V , called **heads**

1. **Linear Projections:** For each head i , project the input into lower-dimensional spaces ($d_k = \frac{d_{\text{model}}}{h}$)

$$Q_i = XW_i^Q, \quad K_i = XW_i^K, \quad V_i = XW_i^V$$

2. **Parallel Attention:** Compute Scaled Dot-Product Attention (SDPA) for each head.

$$\text{head}_i = \text{Attention}(Q_i, K_i, V_i)$$

3. **Concatenation:** Glue the outputs of all h heads back together.

$$H = \text{Concat}(\text{head}_1, \dots, \text{head}_h)$$

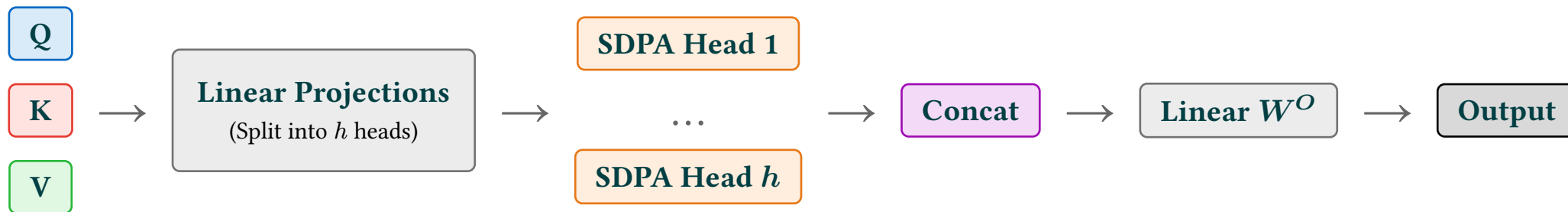
4. **Final Projection:** Apply a final linear (W^O) to mix the information from all heads.

$$\text{Output} = HW^O$$

MHA Architecture Pipeline



南方科技大学



$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h)W^O$$

$$\text{head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V)$$

- $W_i^Q, W_i^K \in \mathbb{R}^{d_{\text{model}} \times d_k}$ (Query & Key projections)
- $W_i^V \in \mathbb{R}^{d_{\text{model}} \times d_v}$ (Value projections)
- $W^O \in \mathbb{R}^{hd_v \times d_{\text{model}}}$ (Output projection)

Grouped Query Attention (GQA) [Bonus Task]



南方科技大学

Motivation: In standard MHA, each Query head has its own Key and Value head.

- **Memory Bandwidth Bound** during inference.
- GQA shares Key/Value heads across multiple Query heads, which reduces memory footprint and improves inference efficiency.

Structure:

- Total of h Query heads, divided into G groups (each group has $\frac{h}{G}$ query heads).
- All queries in a group share exactly **one** Key head and **one** Value head.
- *Multi-Query Attention (MQA)* is the extreme case ($G = 1$) where all h queries share a single KV pair.

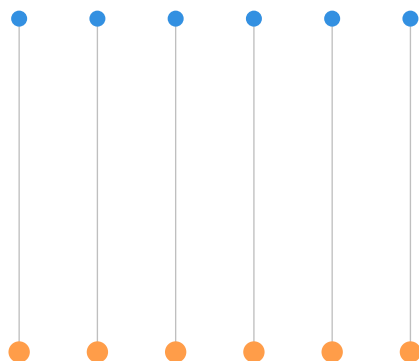
GQA are widely used in modern LLM: Llama 3, Qwen 3, etc.

Comparision of Q/KV Heads for MHA, GQA, MQA



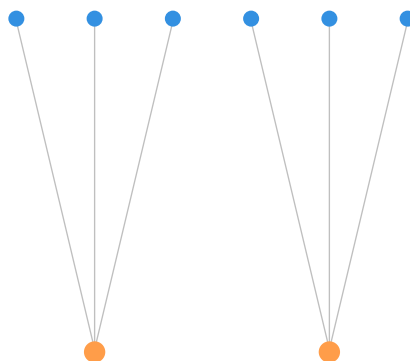
南方科技大学

MHA



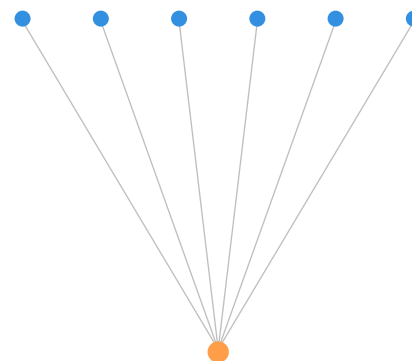
Q Heads: 6
KV Heads: 6

GQA



Q Heads: 6
KV Heads: 2

MQA



Q Heads: 6
KV Heads: 1

Q, K, V Shape Comparison



Q, K, V Shape Comparison (Assuming h total heads, G groups)

Attention Type	Query Shape	Key / Value Shape
Single-Head (SDPA)	(batch, seq, d_k)	(batch, seq, d_k)
Multi-Head (MHA)	(batch, h, seq, d_k)	(batch, h, seq, d_k)
Grouped-Query (GQA)	(batch, h, seq, d_k)	(batch, G, seq, d_k)
Multi-Query (MQA)	(batch, h, seq, d_k)	(batch, 1, seq, d_k)

Lab Tasks

Task 1: Single-Head Scaled Dot-Product Attention (`lab5_sdpa.py`) [50 points]

- Implement attention using only `torch.matmul` and `torch.softmax`
- Include causal mask support (set upper triangle to `-inf`)
- Verify output shapes!

Task 2: Multi-Head Attention Extension (`lab5_mha.py`) [50 points]

- Re-use your SDPA implementation in task 1
- Split into multiple heads using `view` and `transpose`
- Concatenate heads and apply output projection

Task 3: Grouped Query Attention (`lab5_gqa.py`) [Bonus]

- Implement a variation of MHA where multiple queries share a single key/value pair.
- Tip: Use `repeat_interleave` to expand K and V to match Q's head dimension.

Hints for Implementation



For Task 1:

```
1 # Causal mask: prevent attending to future tokens
2 mask = torch.tril(torch.ones(seq_len, seq_len))
3 ...
4 # Q K matching, Question: Why .transpose(-2, -1)?
5 scores = torch.matmul(Q, K.transpose(-2, -1))
6 ...
7 # Apply mask: set masked positions to -inf before softmax
8 scores = scores.masked_fill(mask == 0, float('-inf'))
```

Python

For Task 2:

```
1 # Reshape for multi-head: (batch, seq, d_model) → (batch, heads, seq, d_k)
2 Q = Q.view(batch_size, seq_len, num_heads, d_k).transpose(1, 2)
3 # Concatenate heads: (batch, heads, seq, d_k) → (batch, seq, d_model)
4 output = output.transpose(1, 2).contiguous().view(batch_size, seq_len, d_model)
```

Python

Thanks for Listening!